

What to Retrieve for Effective Retrieval-Augmented Code Generation? An Empirical Study and Beyond

Wenchao Gu
Technical University of Munich
Heilbronn, Germany
wenchao.gu@tum.de

Juntao Chen
The University of Melbourne
Melbourne, Australia
chenjt75@mail2.sysu.edu.cn

Yanlin Wang*
Sun Yat-sen University
Zhuhai, China
yanlin-wang@outlook.com

Tianyue Jiang
Sun Yat-sen University
Zhuhai, China
jiangty9@mail2.sysu.edu.cn

Xingzhe Li
Sun Yat-Sen University
Zhuhai, China
lixzh75@mail2.sysu.edu.cn

Mingwei Liu
Sun Yat-Sen University
Zhuhai, China
liumw26@mail.sysu.edu.cn

Xilin Liu
Huawei Cloud Computing
Technologies Co., Ltd
Shenzhen, China
liuxilin3@huawei.com

Yuchi Ma
Huawei Cloud Computing
Technologies Co., Ltd.
Beijing, China
mayuchi1@huawei.com

Zibin Zheng
Sun Yat-sen University
Zhuhai, China
zhzibin@mail.sysu.edu.cn

Abstract

Repository-level code generation remains challenging due to complex code dependencies and the limitations of large language models (LLMs) in processing long contexts. While retrieval-augmented generation (RAG) frameworks are widely adopted, the effectiveness of different retrieved information sources—contextual code, APIs, and similar snippets—has not been rigorously analyzed. Through an empirical study on two benchmarks, we demonstrate that in-context code and potential API information significantly enhance LLM performance, whereas retrieved similar code often introduces noise, degrading results by up to 15%. Based on the preliminary results, we propose AllianceCoder, a novel context-integrated method that employs chain-of-thought prompting to decompose user queries into implementation steps and retrieves APIs via semantic description matching. Through extensive experiments on public datasets, AllianceCoder achieves state-of-the-art performance, improving Pass@1 by up to 20% over existing approaches.

CCS Concepts

• **Software and its engineering** → **Automatic programming**;
• **Information systems** → **Information retrieval query processing**; • **Computing methodologies** → **Natural language processing**.

Keywords

Code Generation, LLM, Empirical Study

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/26/04
<https://doi.org/10.1145/3744916.3773224>

ACM Reference Format:

Wenchao Gu, Juntao Chen, Yanlin Wang, Tianyue Jiang, Xingzhe Li, Mingwei Liu, Xilin Liu, Yuchi Ma, and Zibin Zheng. 2026. What to Retrieve for Effective Retrieval-Augmented Code Generation? An Empirical Study and Beyond. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773224>

1 Introduction

In modern software development, code generation [28, 45, 46, 63] has emerged as a critical capability to bridge the gap between natural language requirements from developers and executable code. With the advent of large language models (LLMs), state-of-the-art models have demonstrated impressive ability to generate standalone code snippets from natural language queries. However, these achievements are primarily limited to function-level code generation. Repository-level code generation - the task of generating code that with the repository context - remains a challenging task due to the complexity of code dependencies in repositories and inherent limitations of LLMs with long context. To address this challenge, most existing approaches [11, 32, 51, 52, 60] adopt a retrieval-augmented generation (RAG) framework, which retrieves relevant information from the repository, appends it to the query, and then feeds it into LLMs for code generation. Existing works adopts different information sources in the retrieval phase. (1) RepoFormer [52], RepoMinCoder [29], and R²C²-Coder [11] utilize contextual information from the current file to enhance code generation. By incorporating relevant local context and references to third-party libraries, these methods enable LLMs to infer and complete the target function more effectively. (2) RepoFuse [30] extracts relevant APIs by analyzing import statements, while A³CodeGen [32] first generates pseudo-code and then uses it to retrieve potential APIs for code generation. The retrieved API information helps LLMs correctly handle dependencies and ensures that the generated code adheres to API specifications. (3) RepoCoder [60] retrieves code snippets similar to the incomplete code using an context window,

whereas RLCoder [51] applies reinforcement learning to optimize the retrieval process. Providing similar code snippets offers examples of recurring patterns, which aid LLMs in generating logically consistent implementations.

Although these information sources are widely used in RAG-based repository-level code generation, their impact on LLM performance has not been thoroughly studied. This raises a critical question: **In RAG-based repository-level code generation, what information sources truly matter?**

To address this question, we conduct a preliminary study to analyze how different types of retrieved information contribute to improving LLM performance in repository-level code generation. Our experimental results reveal that *contextual information* within the current file and *potential API information* play a crucial role in enhancing LLM performance. Moreover, their combination further improves model performance in repository-level code generation.

Interestingly, contrary to its effectiveness in repository-level code completion, retrieving similar code snippets does not always improve LLM performance in repository-level code generation. In some cases, it can even have a negative impact because there is no guarantee that a functionally similar code snippet exists within the repository, and retrieving dissimilar code snippets may mislead the LLMs, ultimately reducing the quality of the generated code.

Based on these findings, we conclude that retrieving contextual information from both the current file and relevant APIs is essential for enhancing LLM performance in repository-level code generation. While contextual information within the same file is naturally available and does not require additional retrieval techniques, identifying the appropriate APIs to invoke remains a challenge, as LLMs often lack prior knowledge of them.

To address this issue, we propose a simple yet effective approach, AllianceCoder, for retrieving potential APIs. AllianceCoder consists of three key stages: repository API processing, query processing, and context-integrated code generation. **Repository API Processing:** we leverage LLMs to generate natural language descriptions for all APIs within the repository. These descriptions are then encoded into vectors for API retrieval. The motivation behind converting code into natural language is to bridge the semantic gap between programming languages and natural language, thereby improving retrieval accuracy. **Query Processing:** we guide LLMs with examples to decompose the user query into multiple detailed implementation steps. For each step, the model generates descriptions of the potential APIs that may be invoked. The resulting API description sequence is then refined by LLMs and encoded into vectors. **Context-integrated Code Generation:** we utilize these API representation vectors to retrieve the most relevant APIs from the repository based on cosine similarity. The retrieved APIs are then appended to the contextual information and user query before being fed into LLMs for repository-level code generation.

We evaluate our approach on three public datasets and the results show that our two-step retrieval method outperforms state-of-the-art (SOTA) models by up to 20% in terms of the Pass@1 metric.

Our contributions can be summarized as follows:

- We conduct an empirical study to investigate how different retrieved information contribute to LLM performance in repository-level code generation, showing that contextual information and potential APIs play a crucial role in improving performance.
- We propose a simple yet effective approach named AllianceCoder that guides LLMs to decompose functionality into detailed implementation steps and generate descriptions of potential APIs for each step, which are used to retrieve relevant APIs.
- We conduct extensive experiments to evaluate our approach. The results demonstrate that our simple two-step retrieval method significantly outperforms SOTA baselines.

2 Background

2.1 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) enhances generative models by integrating external knowledge [26]. Traditional models, while coherent, often suffer from factual inaccuracies and knowledge gaps due to their reliance on fixed memory [51, 60]. RAG addresses these limitations by incorporating a retrieval mechanism that fetches relevant information from large-scale knowledge sources, such as databases or document corpora, enabling more accurate and contextually grounded outputs.

RAG has been successfully applied in domains requiring factual accuracy and knowledge grounding, including question answering [2, 57], scientific text generation [14], and automated code synthesis [42]. By bridging retrieval-based and generative models, RAG advances the development of more context-aware AI systems.

The RAG process can be formulated as follows:

$$A = \text{Generate}(q, \text{Retrieval}(q, \{c_1, \dots, c_n\})) \quad (1)$$

where q represents the user query, $\{c_1, \dots, c_n\}$ denotes the candidate knowledge set, and A is the output generated by the language model. The function $\text{Retrieval}(\cdot)$ retrieves relevant candidates from the knowledge base based on the given query, while $\text{Generate}(\cdot)$ combines the retrieved information with the original query and feeds it into the language model for response generation.

2.2 Repository-level Code Generation

RAG frameworks have been widely adopted in LLM-based repository-level code generation approaches. The repository-level code generation process can be formulated as follows:

$$\text{Code} = \text{Generate}(q, \text{Retrieval}(q, \text{Codebase})) \quad (2)$$

where Codebase represents the given repository, and q is the user query describing the target function. The function $\text{Retrieval}(\cdot)$ extracts relevant information from the Codebase based on the query, while $\text{Generate}(\cdot)$ integrates the retrieved information with the original query and utilizes LLMs to generate the desired code.

Table 1 summarizes the types of retrieved information used in current LLM-based repository-level code generation approaches. These types can be broadly classified into three categories: similar code, API, and context.

Similar code refers to the similar code snippets retrieved from the same repository. The underlying assumption is that similar code shares certain characteristics with the target code, providing LLMs with useful hints for implementation. From Table 1, we observe that most LLM-based approaches retrieve similar code from the repository, with the exception of A³CodeGen.

API retrieval focuses on identifying relevant APIs that should be invoked in the target code. The goal is to reduce code redundancy

Table 1: Types of retrieved information by different LLM-based approaches.

Approach	Similar Code	API	Context
A ³ CodeGen	✗	✓	✓
RepoCoder	✓	✗	✓
RepoFormer	✓	✗	✓
RepoMinCoder	✓	✗	✓
RLcoder	✓	✗	✓
R ² C ² – Coder	✓	✗	✓
GraphCoder	✓	✗	✓
RepoFuse	✓	✓	✓

while supporting complex functionality. Information about the available APIs helps capture partial functionality within the overall target implementation, improving code integration, reusability, and reducing implementation difficulty. A³CodeGen, and RepoFuse retrieve relevant APIs from the repository.

Context refers to contextual information from the same file as the user query. It includes details such as library imports, class definitions, and member function implementations. This information helps LLMs better understand the code style, maintain consistency, reduce redundancy, and correctly interpret the target functionality. Since contextual information is inherently available within the same file as the user query (which typically corresponds to the target functionality description), all prior LLM-based approaches utilize this information for repository-level code generation.

Limitations. Although current LLM-based code generation approaches widely retrieve such information, their impact on model performance and the optimal way to integrate them remain unclear.

3 Experimental Setup

In this section, we introduce the datasets, baselines and LLMs being evaluated in the experiments, and the evaluation metrics.

3.1 Dataset

There are two key requirements for our benchmark selection. First, it should contain executable test cases. For function-level code generation at the repository level, test-based evaluation better reflects functional correctness and provides a more accurate assessment of model performance compared to metrics like Exact Match or Exact Statement. Second, the benchmark should require customized API invocations in code generation, as this more closely mirrors real-world repository-level software development scenarios and thus better evaluates the model’s practical capabilities.

Based on the aforementioned selection criteria, we select three benchmarks for our evaluation: **CoderEval** [56], **RepoExec** [21] and **ExecRepoBench** [54]. CoderEval is a benchmark used to evaluate code generation performance on pragmatic code generation tasks. It consists of 230 Python and 230 Java tasks from real-world open-source projects. Each task contains a function signature, a function description, a solution, and several unit tests to assess the functional correctness of the generated code. RepoExec is a benchmark designed to evaluate repository-level code completion with complex contextual dependencies. It assesses models’ ability to generate executable and functionally correct code while utilizing

cross-file contexts. Each task provides essential code dependencies specified by developers, along with comprehensive test cases to verify functional correctness. ExecRepoBench is a code completion benchmark comprising 1,200 samples sourced from real-world open-source Python repositories. It employs a multi-level masking strategy guided by abstract syntax trees, masking code at the statement, expression, and function levels to simulate complex multi-file dependencies commonly encountered in real software development. However, this benchmark is not originally designed for the task of code generation. To adapt it for our purpose, we reviewed the 167 test samples from the block completion task in ExecRepoBench and selected those suitable for conversion to full function generation. We then transformed the original block completion tasks into function-level generation tasks that align with our objectives.

To mitigate the risk of data leakage, we conduct the empirical study using only CoderEval and RepoExec, and utilize all three benchmarks solely for evaluating our proposed method.

3.2 Baselines and LLMs evaluated

We explore four state-of-the-art frameworks for repository-level code generation: **RepoFormer** [52], **RepoFuse** [30], **RepoCoder** [60] and **RLCoder** [51]. RepoFormer is a selective RAG framework that enables code language models to retrieve external context only when necessary, improving the efficiency and robustness of repository-level code completion. RepoFuse addresses the Context-Latency Conundrum in repository-level code completion by fusing analogy and rationale contexts with a novel rank truncated generation technique, achieving accurate predictions without sacrificing efficiency. RepoCoder is an iterative retrieval-generation pipeline. It effectively utilizes information scattered across different files within a repository and can generate code at various levels of granularity. RLCoder enhances this approach by implementing a novel reinforcement learning framework that enables the retriever to learn useful content without labeled data. It evaluates retrieved content based on perplexity metrics and includes a stop signal mechanism to determine when to retrieve and which candidates to retain. All frameworks demonstrate significant improvements over baseline methods in repository-level code generation tasks.

3.3 Embedding Model

UniXcoder [17] is a dense retriever, encoding both queries and code snippets into dense vectors. This vectorization mechanism enables the efficient discovery and extraction of semantically aligned code fragments from extensive repositories, based on the computational similarity between their respective vector embeddings.

3.4 Metrics

Since multiple implementations can achieve the same functionality, we adopt *Pass@k* [8] as our sole evaluation metric. In our evaluation, a generated code snippet is considered successful only if it passes all test cases. The *Pass@k* metric represents the success rate after generating code *k* times using LLMs.

3.5 Implementation Details

In all our experiments, we utilized GPT-4o Mini and Gemini 1.5 Flash with the default temperature setting of 0.7. In the preliminary

study on similar code retrieval, we selected the top five most similar code snippets for code generation. For our proposed method, we retrieved the single API from the repository that exhibited the highest cosine similarity to each API description.

4 Preliminary Study

To investigate the impact of different information types on the LLM’s ability to generate repository-level code under the RAG framework, we conduct an preliminary study. In this study, we examine three types of information: contextual information, relevant code (i.e., code similar to the target code), and invoked APIs. To assess the upper-bound performance of each information type, we assume access to perfect information.

- **Contextual information:** This information type includes all text preceding the target function within the same file, such as library import statements, class definitions, and other implemented functions. Providing rich contextual information helps the LLM better understand the target function’s environment. The experimental results for this setting are denoted as $\text{Context}_{\text{LLM}}$.
- **Relevant code:** Building on previous model-based approaches, we leverage LLMs to encode all code candidates within the repository into vector representations. Assuming prior knowledge of the correct target code, we adopt the retrieval framework used in RepoCoder [60], which utilizes a context window to retrieve the top five most relevant code snippets. The experimental results for this setting are denoted as $\text{Similar}_{\text{LLM}}$. Since most previous works retrieve similar code only within the target repository without leveraging any external resources, we adopt the same setting to evaluate the effectiveness of such retrieval approaches, leaving the integration of external RAG as future work.
- **Invoked APIs:** We assume prior knowledge of all APIs invoked within the target code and retrieve them to facilitate repository-level code generation. The retrieved information regarding the invoked APIs comprises their descriptions as well as detailed implementations. The experimental results for this setting are denoted as API_{LLM} .

In our experiments, we evaluate all possible combinations of the three types of retrieved information. Specifically, $\text{ConSim}_{\text{LLM}}$ represents the performance of the LLM when provided with both contextual information and relevant code as input, while $\text{ConAPI}_{\text{LLM}}$ corresponds to the setting where the LLM receives contextual information and the invoked API. Similarly, $\text{SimAPI}_{\text{LLM}}$ denotes the case where the LLM is given relevant code and the invoked API. Finally, $\text{ConSimAPI}_{\text{LLM}}$ represents the setting in which the LLM is provided with all three types of information: contextual information, relevant code, and the invoked API.

To assess the inherent capability of LLMs in repository-level code generation, we also conduct an experiment in which only the user’s original query is fed into the model without any additional information. The results of this setting are denoted as Pure_{LLM} .

Overall, our preliminary study evaluates eight different settings, each corresponding to a unique combination of retrieved information types.

Table 2: Preliminary experimental results on LLM repository-level code generation performance with different retrieved information types (best performance in bold).

Method	RepoExec			CoderEval		
	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
PureGPT	16.62	19.72	23.66	18.26	25.22	26.52
ContextGPT	29.01	34.08	38.59	30.43	36.52	37.83
SimilarGPT	16.90	23.38	24.79	20.00	23.04	23.48
APIGPT	25.63	32.39	34.93	25.65	30.43	33.04
ConSimGPT	22.53	26.48	27.60	14.35	20.43	23.04
ConAPIGPT	37.75	44.51	47.04	36.52	41.30	41.47
SimAPIGPT	25.36	31.55	34.09	22.17	26.52	26.52
ConSimAPIGPT	29.86	38.31	38.87	19.57	23.91	26.52
PureGemini	16.90	20.28	23.10	17.39	21.74	25.22
ContextGemini	30.99	34.08	35.77	23.04	26.09	27.39
SimilarGemini	17.18	22.25	23.38	13.48	16.52	16.96
APIGemini	26.20	30.99	34.93	22.61	24.78	26.09
ConSimGemini	24.22	27.33	29.02	13.91	15.22	15.22
ConAPIGemini	38.59	41.69	44.51	25.22	27.39	28.70
SimAPIGemini	27.32	31.27	32.96	19.13	22.17	22.61
ConSimAPIGemini	32.67	36.90	38.59	22.17	23.04	23.91

Table 3: Preliminary Results on LLM-Based Repository-Level Code Generation Using Different Retrieval Types Across Repository Scales on CoderEval.

Method	Scale	Pass@1	Pass@3	Pass@5
SimilarGPT	large	20.00	22.90	23.23
	medium	17.59	20.37	20.37
	small	24.39	29.27	31.71
ConSimGPT	large	15.16	21.61	23.87
	medium	9.26	13.89	15.74
	small	19.51	26.83	34.15
SimAPIGPT	large	21.61	25.81	25.81
	medium	19.44	24.07	24.07
	small	31.71	36.59	36.59
ConSimAPIGPT	large	19.68	23.55	26.13
	medium	20.37	24.07	25.93
	small	17.07	26.83	31.71

5 Results of Preliminary Study

In this section, we aim to answer the following research questions(RQs):

- **RQ1: How does each type of information contribute to performance improvement in repository-level code generation?**
- **RQ2: Can the performance gains from contextual information and invoked APIs be fully captured by simply integrating them?**
- **RQ3: Are the performance improvements from contextual information and invoked APIs independent of each other?**

Table 4: The Impact of Information Order Changes on the Performance with GPT.

Method	RepoExec			CoderEval		
	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
Con + API _{GPT}	37.75	44.51	47.04	36.52	41.30	41.47
API + Con _{GPT}	35.12	44.75	47.59	35.65	40.87	40.87

5.1 RQ1: How does each type of information contribute to performance improvement in repository-level code generation?

Table 7 presents the experimental results of our preliminary study. As shown in the table, contextual information provides the most significant performance improvement for LLMs in repository-level code generation. This result is expected, as contextual information supplies critical supplementary information, including imported libraries, class definitions, and implementations of other member functions. These elements enrich the user query with additional context, enabling LLMs to better understand the intended functionality and generate more accurate code.

Moreover, we observe that incorporating invoked API also substantially enhances performance. This improvement stems from two key factors. First, explicitly specifying relevant APIs within the repository allows LLMs to invoke them directly rather than reimplementing functionality from scratch, thereby reducing the likelihood of incorrect implementations. Second, API chains provide a structured reference, guiding LLMs to follow an established logical flow, which simplifies the code generation process.

Furthermore, the combination of both contextual information and invoked API yields the best overall performance. From a human developer’s perspective, these elements collectively provide all the essential knowledge required for repository-level code generation, enabling more efficient implementation of the target functionality. Similarly, LLMs can leverage this prior knowledge to enhance their generative capabilities. Given that modern LLMs already excel at function-level code generation, supplying contextual information and API usage information effectively decomposes the complex task of repository-level code generation into smaller, well-defined function-level subtasks. This decomposition allows LLMs to fully utilize their strengths in function-level generation, leading to more accurate and reliable repository-level code generation.

However, an unexpected finding is that the retrieved relevant code contributes very little to performance improvement and can even degrade performance when combined with other types of supplementary information. For instance, in both datasets, the model incorporating all three types of information (contextual information, invoked APIs, and relevant code) performs significantly worse than the model that utilizes only contextual information and invoked APIs. Notably, in the CodeEval dataset, the model leveraging all three types of information underperforms even when compared to models using only contextual information or only invoked APIs. In fact, its performance is close to that of LLMs without any retrieved information at all.

Most prior retrieval-augmented approaches have focused on repository-level code completion, where the task is to complete

Table 5: Statistical Results of Input Prompt Length.

LLM	Probability of Superiority		Mann–Whitney U test	
	RepoExec	CoderEval	RepoExec	CoderEval
GPT	0.556	0.616	0.254	0.089
Gemini	0.672	0.791	0.014	< 0.001

partially written code. In such settings, the required generation is typically short, making it easier to retrieve code snippets exhibiting similar patterns based on the surrounding context and partial functionality, which can effectively guide LLMs. In contrast, repository-level code generation tasks provide only a high-level functional description and function signature as input, requiring the model to generate an entire function. This scenario presents greater challenges for retrieval: the limited input context and increased complexity of the target functionality make it significantly harder to retrieve truly relevant code snippets. Consequently, retrieved snippets may offer limited benefit or, worse, introduce misleading signals that adversely affect generation quality.

Our analysis substantiates this risk. For instance, when generating the `unit_of_work` decorator in `neo4j/work/query.py`, which manages transaction execution via metadata and timeout configurations, the most similar retrieved snippet—although located in the same module—was functionally unrelated, focusing instead on query result summaries (`ResultSummary`, `SummaryCounters`). This example highlights a limitation of retrieval-augmented code generation: high lexical or structural similarity does not necessarily imply functional relevance. When LLMs rely on such mismatched examples, their performance can degrade rather than improve.

To examine whether repository scale affects the contribution of similar code retrieval in repository-level code generation, we analyze performance across datasets of varying sizes. Specifically, we divide repositories into three groups: small (<5,000 lines), medium (5,000–20,000 lines), and large (>20,000 lines).

As the performance trends are similar across different LLMs and datasets, we report only the results on CoderEval using GPT for brevity. Table 3 presents the performance of GPT with similar code retrieval across these repository sizes. Results indicate that LLMs attain optimal performance with small target repositories, suggesting that LLMs may more easily implement the target function in compact repository contexts.

In addition, we also investigate the impact of the order of API and contextual information on the performance with GPT. Table 4 shows that changing the order does not lead to any significant performance differences.

Finding 1: Contextual information and invoked APIs contribute significantly to performance improvements. However, the inclusion of similar code does not always lead to improvements and, in some cases, can even negatively impact performance in repository-level code generation.

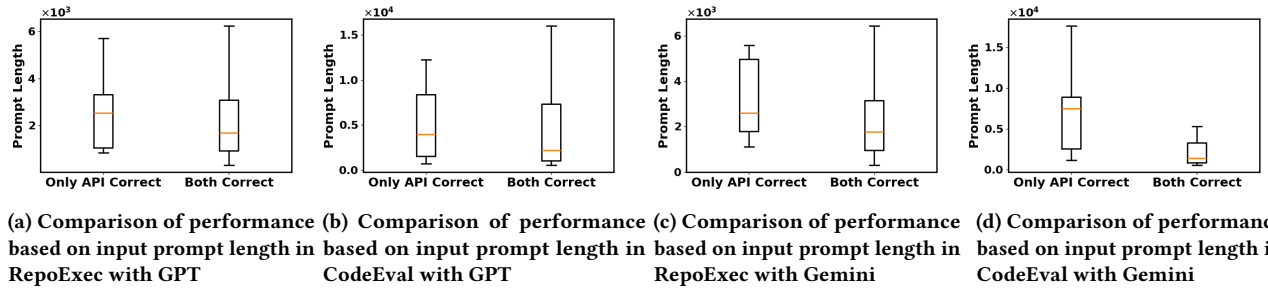
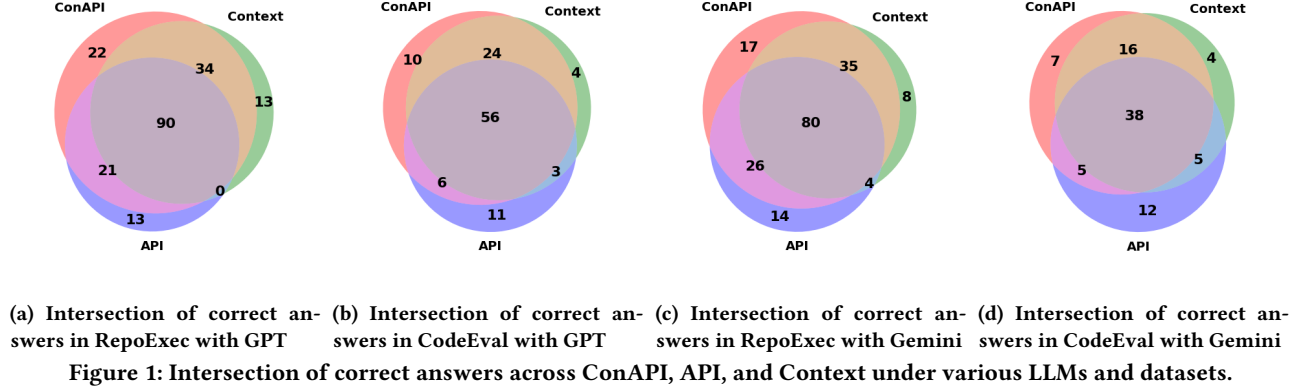


Figure 2: Comparison of Input Prompt Lengths for Test Cases: Success in Both ConAPI & API vs. API-Only Success Across Different Datasets and LLM.

Table 6: Performance with Varying Prompt Lengths.

LLM	Length	RepoExec			CoderEval		
		Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
API _{GPT}	long	24.00	33.00	34.00	18.95	23.53	24.84
	medium	27.47	32.97	37.36	28.26	32.61	39.13
	short	25.93	32.10	34.57	54.84	61.29	64.52
API _{Gemini}	long	19.00	27.00	34.00	17.65	18.95	19.61
	medium	34.07	41.76	41.76	21.74	26.09	30.43
	short	24.69	26.54	30.25	48.39	51.61	51.61
ConAPI _{GPT}	long	36.00	39.00	41.00	31.37	36.60	37.25
	medium	39.56	46.15	49.45	45.65	50.00	50.00
	short	38.27	47.53	50.00	48.39	51.61	51.61
ConAPI _{Gemini}	long	37.00	38.00	40.00	18.95	19.61	20.26
	medium	42.86	46.15	47.25	32.61	34.78	36.96
	short	37.65	41.98	46.30	45.16	51.61	54.84

5.2 RQ2: Can the performance gains from contextual information and invoked APIs be fully captured by simply integrating them?

To further assess the effectiveness of integrating contextual information with invoked APIs, we analyze the intersection of test-passing generations—i.e., generated code that passes the corresponding test cases for each query—among $\text{Context}_{\text{LLM}}$, API_{LLM} , and $\text{ConAPI}_{\text{LLM}}$.

To mitigate the impact of randomness on our experimental results, we evaluate the intersection of generations that pass the test cases using the Pass@5 metric, where each query is tested with five generated code candidates. The results are presented in Figure 1.

From the figure, we observe that $\text{ConAPI}_{\text{LLM}}$ successfully covers most of the test-passing generations in either API_{LLM} or $\text{Context}_{\text{LLM}}$. This result is intuitive, as $\text{ConAPI}_{\text{LLM}}$ integrates both contextual and API-related information. If the retrieved contextual and API-related information are indeed effective, then $\text{ConAPI}_{\text{LLM}}$ should naturally have the same test-passing generations as $\text{Context}_{\text{LLM}}$ and API_{LLM} .

However, we also observe that, given the same query, the code generated by $\text{Context}_{\text{LLM}}$ or API_{LLM} can successfully pass the test cases, but the generation from $\text{ConAPI}_{\text{LLM}}$ fails. One possible explanation is the inherent randomness in the model’s output. Despite testing each case five times, some degree of randomness persists, leading to minor inconsistencies in the results. The number of cases where the code generated by $\text{Context}_{\text{LLM}}$ passes the test cases but the code generated by $\text{ConAPI}_{\text{LLM}}$ fails is relatively small, suggesting that model randomness might be the primary cause.

In contrast, Figure 1b, Figure 1c, and Figure 1d reveal a notable number of cases where the code generated by API_{LLM} passes the test cases but the code generated by $\text{ConAPI}_{\text{LLM}}$ fails. These cases correspond to the combined purple and blue sections in the figures, with counts of 14, 18, and 17, respectively. This discrepancy cannot

be solely attributed to model randomness, indicating that other factors may be influencing the performance.

To further investigate this issue, we analyze the input prompt lengths for two groups of queries: those for which the code generated by both API_{LLM} and $\text{ConAPI}_{\text{LLM}}$ passes the test cases, and those for which only the code from API_{LLM} passes. The results are shown in Figure 2. From this figure, we observe a significant difference in input prompt length between two groups of queries: those for which only the code generated by API_{LLM} passes the test cases, and those for which both API_{LLM} and $\text{ConAPI}_{\text{LLM}}$ succeed.

Besides, we conduct experiments using the Probability of Superiority [16] and the Mann–Whitney U statistic [38]. Specifically, we compare the prompt lengths of cases that passed with API but failed with API+Context to those that passed with either API or API+Context. The results are shown in Table 5. From these tables, we observe that the prompt length for cases that passed with API but failed with API+Context tends to be longer than those that succeeded with either configuration, although the extent of this difference varies. In the case of GPT, the difference shows a slight advantage but is not statistically significant. In contrast, for Gemini, the difference is statistically significant.

What’s more, We analyzed our model’s performance across prompt lengths, categorized as long (>10,000 tokens), medium (5,000–10,000 tokens), and short (<5,000 tokens), as shown in Table 6. Generally, performance decreases as prompt length grows, aligning with our empirical observations. An exception appears in the RepoExec dataset, where medium prompts improve results. This suggests a trade-off: longer prompts provide richer context for code generation but may also introduce irrelevant information that distracts the model and reduces performance.

Since contextual information is naturally connected to the user query, we structure the prompt by first providing relevant API information, followed by contextual information along with the user query. This ordering results in API-related content appearing farther from the user query in the prompt. While modern LLMs can handle long inputs, content appearing earlier in the prompt may have a diminished impact on later generated responses if the input length is too extensive.

Finding 2: Naively appending invoked APIs with excessive contextual information may cause queries that would otherwise yield correct code (i.e., code that passes the test cases) to instead produce incorrect outputs, due to the increased input length.

5.3 RQ3: Are the performance improvements from contextual information and invoked APIs independent of each other?

Since the contextual information contains numerous member functions, some of which may be the invoked functions, it is possible that the contextual information already includes API details. To better understand how different types of information affect model performance in repository-level code generation, we analyze the number of test-passing generation by $\text{Context}_{\text{LLM}}$ under two conditions: (1) when the contextual information fully contains the

invoked APIs and (2) when it does not contain any APIs at all. The statistical results are presented in Table 7.

In this table, Fully Contained refers to cases where the contextual information includes all invoked API details, while Not Included indicates cases where the contextual information does not contain any invoked APIs. The metric CPass represents the number of queries for which the code generated by $\text{Context}_{\text{LLM}}$ passes the test cases, whereas BPass denotes the number of queries for which both $\text{Context}_{\text{LLM}}$ and API_{LLM} generate code that passes the test cases. The percentage under CPass indicates the success rate—i.e., the proportion of queries whose generated code passes all test cases—under each condition (Fully Contained or Not Included). The percentage under BPass reflects the proportion of successful generations by $\text{Context}_{\text{LLM}}$ that are also successful when using API_{LLM} . To mitigate the effect of model randomness, each query is evaluated five times. To isolate the effect of the LLMs’ intrinsic code generation capabilities, we exclude queries whose test cases can be trivially passed by code generated from Pure_{LLM} .

From Table 7, we observe that API information within the contextual data significantly improves performance. Specifically, by comparing cases where the contextual information fully contains the invoked APIs to those where it does not contain any, we find that $\text{Context}_{\text{LLM}}$ achieves a notable performance boost in both datasets. In the CoderEval dataset, the performance is even doubled.

Moreover, the queries for which both $\text{Context}_{\text{LLM}}$ and API_{LLM} generate code that passes the test cases exhibit strong similarity. Notably, when the contextual information fully contains the invoked APIs, approximately 50–70% of the queries successfully handled by $\text{Context}_{\text{LLM}}$ are also successfully handled by API_{LLM} . In this scenario, the invoked API information is essentially a subset of the contextual information, suggesting that comparable performance can be achieved by reducing the complexity of contextual information to only the invoked API details. This finding highlights the critical role of invoked APIs in repository-level code generation.

Additionally, we acknowledge that contextual information still provides valuable content beyond the invoked APIs. This is evident from the fact that $\text{Context}_{\text{LLM}}$ maintains a certain level of test case success even when it does not contain any invoked APIs, while pure API information alone is insufficient for generating correct code. This indicates that contextual information contributes additional useful signals to the code generation process.

Finding 3: Contextual information sometimes includes the invoked APIs for the target code, and a portion of the performance improvement attributed to contextual information actually stems from the presence of these invoked APIs.

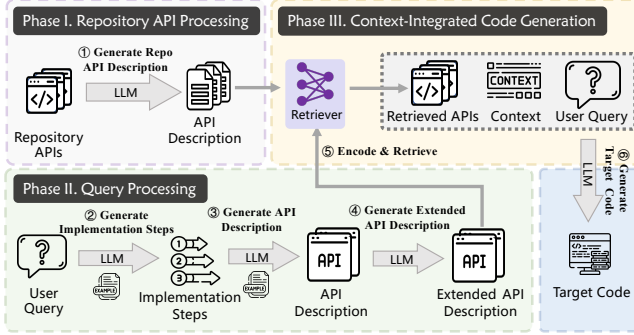
6 AllianceCoder

6.1 Overview

Based on our empirical study, potential API retrieval plays a crucial role in repository-level code generation, yet identifying which APIs can be invoked within a repository remains a challenging problem. To address this, we propose AllianceCoder, a simple yet effective approach for retrieving relevant APIs. As illustrated in Figure 3, AllianceCoder consists of three phases: repository API processing, query processing, and context-integrated code generation. In the

Table 7: Statistics of test-passing generation: comparison by context alone vs. context with API under different API containment conditions.

LLM	RepoExec				CodeEval			
	Fully Contained		Not Included		Fully Contained		Not Included	
	CPass	BPass	CPass	BPass	CPass	BPass	CPass	BPass
GPT	30 (28.0%)	18 (60.0%)	36 (24.5%)	12 (33.3%)	8 (47.1%)	5 (62.5%)	29 (20.3%)	8 (27.6%)
Gemini	22 (20.6%)	15 (68.2%)	25 (16.7%)	8 (32.0%)	4 (22.2%)	2 (50.0%)	15 (10.3%)	2 (13.3%)

**Figure 3: AllianceCoder framework.**

repository API processing phase, we leverage LLMs to generate natural language descriptions for each API in the repository and encode them into representation vectors using pre-trained models. During query processing, we guide LLMs with examples to generate descriptions of potentially invoked API functionalities, which are similarly encoded into vectors. Finally, in the context-integrated code generation phase, we retrieve relevant APIs for each API description based on the cosine similarity between their vector representations. The examples and prompts used in AllianceCoder are provided in the Appendix.

6.2 Repository API Processing

6.2.1 Generate Repo API Description. In our proposed approach, API retrieval is performed using vector cosine similarity, with the encoding process handled by a pre-trained model in an unsupervised manner. However, a semantic gap exists between programming languages and natural language, which can result in low cosine similarity between code snippets and their corresponding descriptions. To mitigate this issue, we first leverage LLMs to generate natural language descriptions for each API in the repository. The process of generating these descriptions can be formulated as follows:

$$P(D | C) = \prod_{t=1}^T P(d_t | d_{<t}, C) \quad (3)$$

where D represents the code description, C denotes the given code, T is the length of the description, and $P(\cdot)$ represents the probability distribution.

Once the descriptions for all APIs are generated, they are encoded into representation vectors and stored in advance for efficient retrieval.

6.3 Query Processing

6.3.1 Generate Implementation Steps. The goal of the query processing phase is to generate descriptions of the APIs that may be invoked in the target function. To achieve this, we first provide examples and instruct LLMs to decompose the overall code task into a sequence of concrete implementation steps. This process can be formulated as follows:

$$\{s_1, \dots, s_n\} = \text{Generate}(q, \{e_1, \dots, e_m\}) \quad (4)$$

where $\{s_1, \dots, s_n\}$ represents the detailed implementation steps generated by LLMs, q is the user query specifying the target code functionality, $\{e_1, \dots, e_m\}$ represents the set of examples manually crafted by us for in-context learning, and $\text{Generate}(\cdot)$ is the function that integrates the examples and user query to generate the implementation steps.

6.3.2 Generate API Description. Once these steps are obtained, they are fed back into the LLM along with additional examples and prompts to generate descriptions of the potential APIs used in each step. This process is expressed as follows:

$$\{a_1, \dots, a_o\} = \text{Generate}(\{e_1, \dots, e_m\}, \{s_1, \dots, s_n\}) \quad (5)$$

where $\{s_1, \dots, s_n\}$ are the previously generated implementation steps, $\{a_1, \dots, a_o\}$ represents the descriptions of APIs that may be used in these steps, $\{e_1, \dots, e_m\}$ represents the set of examples manually crafted by us for in-context learning, and $\text{Generate}(\cdot)$ is the function that integrates the examples and implementation steps to generate API descriptions.

6.3.3 Generate Extended API Description. However, LLMs may sometimes generate API descriptions that encompass composite functionalities, requiring multiple APIs for full implementation. To address this issue, we further guide LLMs to expand the set of potential APIs, ensuring broader coverage and reducing the risk of omitting relevant APIs. This expansion process is formulated as follows:

$$\{a'_1, \dots, a'_k\} = \text{Generate}(\{a_1, \dots, a_o\}) \quad (6)$$

where $\{a_1, \dots, a_o\}$ are the previously generated potential API descriptions, $\{a'_1, \dots, a'_k\}$ represents the expanded set of API descriptions, and $\text{Generate}(\cdot)$ is the function that refines and extends the API descriptions based on the initial candidates.

6.4 Context-Integrated Code Generation

6.4.1 Encode & Retrieve. Once the potential API descriptions are generated, they are encoded into representation vectors using the same pre-trained model applied for encoding API descriptions in the repository. API retrieval is performed by computing the cosine similarity between the generated potential API description derived

from the query and the existing API descriptions in the repository, returning the one with the highest similarity score. Each potential API description generated from the query retrieves one relevant API from the repository.

6.4.2 Generate Target Code. Finally, all retrieved APIs are appended to the contextual information and the user’s query before being fed into LLMs.

7 Evaluation

We aim to answer the following research questions (RQs):

- **RQ4: How effective is AllianceCoder in repository-level code generation?**
- **RQ5: How does AllianceCoder perform in API prediction, and what is the gap between its predictions and the ideal outcomes?**
- **RQ6: How effective is generating natural language descriptions for API retrieval compared to using code snippets?**

7.1 RQ4: How effective is AllianceCoder in repository-level code generation?

Table 8 presents a performance comparison of different approaches across various LLMs. As a baseline, we include Context_{LLM} from our preliminary study. The results demonstrate that AllianceCoder consistently achieves state-of-the-art (SOTA) performance across all evaluation metrics on both datasets.

Notably, AllianceCoder achieves a significant improvement in the Pass@1 metric, outperforming the best-performing baselines (For instance, the best baseline with GPT on the Repoexec dataset is RLCoder) by approximately 2–20%. Since Pass@1 reflects the success rate on the first attempt, this enhancement significantly reduces the need for multiple regeneration attempts, thereby improving the overall user experience.

Furthermore, the performance gains of AllianceCoder are higher compared to Context_{LLM} and approach those of ConAPI_{GPT} on the CoderEval dataset. This result underscores the effectiveness of API retrieval in AllianceCoder. However, it is important to note that the performance improvement of AllianceCoder on the CoderEval dataset with Gemini is relatively limited. The limited potential improvement from API retrieval primarily stems from the inherently low upper bound, as defined by the performance of ConAPI_{LLM}, which includes all required APIs and thus reflects an ideal retrieval condition. Even under optimal API retrieval conditions, the maximum achievable improvement in Pass@3 and Pass@5 remains around 5%. We attribute this limitation to the intrinsic generation capabilities of the LLM, which constrain the extent to which API retrieval can enhance performance.

Summary 1: AllianceCoder achieves SOTA performance across all baselines, with particularly notable improvements in Pass@1. On the CoderEval dataset with GPT, AllianceCoder’s performance closely approaches the theoretical upper bound of API retrieval.

7.2 RQ5: How does AllianceCoder perform in API prediction, and what is the gap between its predictions and the ideal outcomes?

To evaluate the accuracy of API descriptions generated by LLMs, we randomly sampled 120 cases (20 per LLM per benchmark) and manually verified if each description correctly matched the given API. These results indicate that AllianceCoder generates approximately 50% of correct potential API descriptions.

To evaluate the effectiveness of AllianceCoder in API prediction, we examine the API recall numbers obtained by AllianceCoder, as shown in Table 10. In this table, Higher denotes the percentage of queries for which AllianceCoder recalls more APIs than are actually required by the target function. Equal refers to the percentage of queries where the number of recalled APIs exactly matches the actual requirement, and Lower corresponds to the percentage of queries where AllianceCoder recalls fewer APIs than needed. From the table, we observe that LLMs tend to generate more APIs than the target function actually requires. Additionally, different LLMs exhibit varying tendencies; for example, GPT tends to recall more APIs compared to Gemini.

Table 11 presents the API recall ratios for AllianceCoder and AllianceCoder combined with context. In this table, Recall represents the API recall ratio of AllianceCoder across all queries, BRecall denotes the API recall ratio for queries whose generated code passes the test cases under both AllianceCoder and ConAPI_{LLM} (i.e., the ideal condition with API retrieval), and CRecall represents the API recall ratio of AllianceCoder including APIs invoked in the contextual information. The results show that the API recall ratio for queries with passing generated code (BRecall) is higher than the overall recall ratio of AllianceCoder (Recall), indicating that providing correct potential APIs as input can improve performance. However, even when considering APIs embedded in contextual information, the API recall ratio remains relatively low. Nevertheless, the overall performance of AllianceCoder is comparable to the ideal performance achieved by ConAPI_{LLMs}, as reported in Table 8 on the CoderEval dataset. While AllianceCoder exhibits a low actual API recall ratio in Table 11, its performance is comparable to that of ConAPI_{LLMs} (a scenario with perfect API recall) as shown in Table 8—indicating that even partial API recall can substantially enhance overall model performance.

To investigate whether task decomposition in AllianceCoder contributes to more accurate API descriptions and ultimately improves its performance, we conduct an ablation study by removing the task decomposition step. Instead, we directly prompt the LLM (GPT) to generate the API descriptions. The results are presented in Table 12. As shown in the table, the model’s performance declines, particularly in the Pass@1 metric, highlighting the effectiveness of task decomposition in AllianceCoder.

Summary 2: AllianceCoder tends to predict more APIs than actually required, and providing LLMs with a subset of potential invoked APIs can also enhance repository-level code generation performance.

Table 8: Performance comparison of different approaches with various LLMs (best performance in bold).

Approach	RepoExec			CoderEval			ExecRepoBench		
	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
RepoCoder _{GPT}	27.20	35.69	38.24	18.70	20.87	22.17	19.16	19.16	19.76
RLCoder _{GPT}	31.55	36.62	38.87	16.52	19.13	21.30	22.75	31.14	31.74
RepoFormer _{GPT}	28.61	37.68	38.81	16.52	19.13	19.13	10.78	13.77	15.57
RepoFuse _{GPT}	29.75	36.26	38.52	17.39	19.57	21.74	23.95	26.95	28.74
Context _{GPT}	29.01	34.08	38.59	30.43	36.52	37.83	29.34	32.93	34.73
AllianceCoder _{GPT}	34.93 (10.7%↑)	40.28 (6.9%↑)	41.97 (8.0%↑)	36.52 (20.0%↑)	40.00 (9.5%↑)	41.30 (9.2%↑)	32.34 (10.2%↑)	34.73 (5.5%↑)	35.33 (1.7%↑)
ConAPI _{GPT}	37.75	44.51	47.04	36.52	41.30	41.74	35.93	39.52	41.92
RepoCoder _{Gemini}	25.63	31.26	32.95	13.91	15.65	16.09	11.38	14.97	17.37
RLCoder _{Gemini}	12.68	15.21	18.31	12.17	12.61	13.04	28.14	31.14	32.34
RepoFormer _{Gemini}	8.50	16.71	20.68	10.43	12.17	12.17	14.37	22.75	25.75
RepoFuse _{Gemini}	29.18	33.42	35.41	15.22	16.52	17.39	17.96	22.16	22.75
Context _{Gemini}	30.99	34.08	35.77	23.04	26.09	27.39	25.75	26.35	28.74
AllianceCoder _{Gemini}	35.49 (14.5%↑)	39.16 (14.9%↑)	41.13 (15.0%↑)	24.78 (7.6%↑)	26.52 (1.6%↑)	27.82 (1.6%↑)	28.74 (2.1%↑)	32.34 (3.9%↑)	32.93 (1.9%↑)
ConAPI _{Gemini}	38.59	41.69	44.51	25.22	27.39	28.70	31.74	34.13	34.13

Table 9: Ideal Recall Performance of API Descriptions Generated by LLMs.

Method	RepoExec	CoderEval	ExecRepoBench
GPT	0.44	0.39	0.65
Gemini	0.30	0.31	0.60

7.3 RQ6: How effective is generating natural language descriptions for API retrieval compared to using code snippets?

Although most existing pre-trained models attempt to unify programming languages and natural language modalities during the pre-training stage in an unsupervised manner, a persistent alignment gap remains between these two modalities. This misalignment results in representation vectors of code-query pairs that do not align well, which serves as the primary motivation for AllianceCoder’s approach—first generating natural language descriptions for APIs before retrieval.

To examine the impact of this semantic gap on overall performance, we conduct an experiment in which APIs from the repository are directly encoded into representation vectors, followed by retrieval based on these vectors. The results, summarized in Table 13, yield several key observations.

First, performance is highly sensitive to the specific implementation details of code within the repository. The performance degradation in the CoderEval dataset is significantly more pronounced than in the RepoExec dataset for both LLMs. We attribute this to the greater functional complexity and diverse coding styles in CoderEval, which exacerbate the semantic gap between code and its corresponding natural language descriptions.

Additionally, the ability of LLMs to generate accurate and semantically meaningful descriptions plays a crucial role in retrieval performance. Different LLMs exhibit varying degrees of performance degradation within the same dataset. For instance, in RepoExec, GPT suffers a smaller performance drop compared to Gemini, whereas in CoderEval, GPT performs worse. This discrepancy likely arises from differences in description generation styles. Since each LLM employs distinct training strategies and datasets, their output formats vary, ultimately influencing retrieval effectiveness.

Summary 3: Directly using code for retrieval leads to a decline in AllianceCoder’s overall performance. The extent of this performance drop varies depending on the choice of LLM and dataset.

8 Related Works

8.1 Code Generation

The task of repository-level code generation is gaining significant attention for intelligent software development in real-world scenarios [34, 46, 50, 59]. Traditional code generation methods can be primarily categorized into rule-based approaches [23], statistical probability model-based methods [43], and deep learning-based approaches [5, 9, 24, 50]. With the advancement of large language models [1, 18, 33, 44, 53], many researchers have introduced LLMs into code generation tasks [19, 27, 55, 65]. In combination with LLMs, numerous studies have adopted Retrieval-Augmented Generation techniques for code completion and code generation tasks [22, 37, 40, 58, 64].

For example, ReCode [22] improves neural code generation via subtree retrieval. RedCoder [40] retrieves top-k candidate codes based on functionality descriptions, aggregates them, and then generates the target code. DocPrompting [64] leverages code documentation to handle unknown functions and library code. ReACC [37] retrieves similar code using a weighted score combining cosine similarity and BM25, concatenating results with incomplete code as input to the LLM. These studies show that integrating LLMs with RAG techniques significantly boosts code completion and generation. APICoder [58] trains models using API documentation to better generate private library code.

8.2 Repository-level Code Generation

Repository-level code generation leverages the extensive context within code repositories and has become a key research focus [13, 15, 36, 41, 51, 60]. RepoCoder [60] and De-Hallucinator [15] use iterative retrieval and generation. RepoMinCoder [29] enhances retrieval-augmented generation by adding a screening and ranking step. R2C2-Coder [11] builds a candidate pool and retrieves relevant content for each completion position. RepoGraph [39]

Table 10: Comparison of API recall counts: AllianceCoder vs. actual API counts.

Approach	RepoExec			CoderEval			ExecRepoBench		
	Higher	Equal	Lower	Higher	Equal	Lower	Higher	Equal	Lower
AllianceCoder _{GPT}	74.45	6.90	18.61	72.64	4.70	22.64	80.41	4.73	14.86
AllianceCoder _{Gemini}	51.42	13.25	35.33	58.49	9.90	31.60	59.74	6.49	33.77

Table 11: Comparison of API recall ratios for AllianceCoder and AllianceCoder with context combination.

Approach	RepoExec			CoderEval			ExecRepoBench		
	Recall	BRecall	CRecall	Recall	BRecall	CRecall	Recall	BRecall	CRecall
AllianceCoder _{GPT}	20.38	21.33	29.23	16.81	24.00	46.22	45.17	47.32	50.28
AllianceCoder _{Gemini}	14.62	16.05	23.46	10.08	15.38	39.50	42.16	45.12	47.82

Table 12: The Effectiveness of Task Decomposition on the Performance of AllianceCoder Using GPT.

Order	RepoExec			CoderEval			ExecRepoBench		
	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
AllianceCoder	34.93	40.28	41.97	36.52	40.00	41.30	32.34	34.73	35.33
AllianceCoder _{w/o Decomp}	33.99 (2.7%↓)	39.66 (1.5%↓)	41.64 (0.8%↓)	32.60 (10.7%↓)	38.26 (4.3%↓)	39.13 (5.3%↓)	27.54 (14.8%↓)	31.73 (8.6%↓)	33.53 (5.1%↓)

Table 13: Performance comparison between text-to-text and text-to-code retrieval.

Approach	RepoExec			CoderEval			ExecRepoBench		
	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
AllianceCoder _{GPT}	34.93	40.28	41.97	36.52	40.00	41.30	32.34	34.73	35.33
AllianceCoder(Code) _{GPT}	33.71 (3.5%↓)	39.38 (2.2%↓)	41.93 (0.1%↓)	23.47 (35.7%↓)	25.65 (35.9%↓)	26.96 (34.7%↓)	25.15 (22.2%↓)	29.34 (15.5%↓)	29.94 (15.3%↓)
AllianceCoder _{Gemini}	35.49	39.16	41.13	24.78	26.52	27.82	28.74	32.34	32.93
AllianceCoder(Code) _{Gemini}	28.45 (19.8%↓)	33.80 (13.7%↓)	34.65 (15.8%↓)	18.26 (26.3%↓)	20.43 (23.0%↓)	21.30 (23.4%↓)	26.95 (6.3%↓)	28.74(11.1%↓)	30.54(7.3%↓)

models repositories as graphs to improve analysis. Rambo [7] incorporates repository-specific elements for accurate method generation. DraCo [10] applies extended dataflow analysis to create repo-specific context graphs for precise retrieval.

CodePlan [4], RepoFuse [30], and A³CodeGen [32] leverage static code analysis to retrieve relevant code candidates. Fixed-window methods risk losing semantics, and dependency parsing is limited by graph context. To address these, RLCoder [51] and RepoGenReflex [48] adopt reinforcement learning. CoCoGen [6] uses compiler feedback and static analysis to fix mismatches between generated code and project context. RepoGenix [31] combines relevant contexts with Context-Aware Selection to compress prompts.

In methods like KNN-LM [25], KNN-LM [47], and FT2Ra [20], retrieval is triggered for each generated token, causing retrieval time to grow with sequence length. CodeAgent [61] and ToolGen [49] explore integrating tool invocation mechanisms. Benchmarks such as RepoEval [60], RepoBench [35], and CrossCodeEval [12] systematically evaluate code generation across contexts. Some works study selective retrieval [52, 62]; RepoFormer [52] improves repository-level generation by deciding when to retrieve, while Probing-RAG [3] analyzes hidden states to adaptively trigger retrieval.

In summary, these approach proves to be effective for repository-level code generation. Nevertheless, identifying which APIs can be invoked within a repository remains a challenging problem.

9 Threats to Validity

Internal Threats. First, our evaluation is limited to Python, which may affect generalizability to other languages. We used real-world benchmarks (CoderEval, RepoExec) and plan to test more languages in future work.

Second, due to computational constraints, we evaluated only two LLMs, which may not cover all state-of-the-art models. These models were chosen for their prominence in code generation research [7, 39, 61], and our method is model-agnostic.

Third, Pass@k measures functional correctness but not code quality aspects like readability or maintainability. Future work could incorporate static or human evaluation.

Fourth, potential data leakage exists since some repository code may appear in LLM pretraining data. Using dynamically evolving benchmarks may help mitigate this.

External Threats. First, inherent randomness in LLM outputs remains despite setting temperature to 0.7. We mitigated this by repeating experiments five times and averaging results.

Second, our retrieval depends on unsupervised API description generation, assuming APIs have meaningful natural language semantics. This may underperform on poorly documented or cryptically named APIs. We partially address this with LLM-generated descriptions, but hallucination risks require further study.

10 Conclusion

This paper presents an empirical study on the role of retrieved information in retrieval-augmented repository-level code generation.

Empirical results demonstrate that in-context code and potential API information significantly enhance LLM performance, while retrieved similar code snippets often introduce noise. Based on these insights, we propose AllianceCoder, a novel context-integrated method that leverages chain-of-thought prompting to decompose requirements and retrieve APIs via semantic descriptions according to the decomposed requirements. Experiments on multiple benchmarks show that AllianceCoder outperforms state-of-the-art baselines by up to 20% in Pass@1, highlighting the importance of targeted retrieval strategies for complex code generation.

Acknowledgments

This work is supported by CCF-Huawei Populus Grove Fund CCF-HuaweiSE202403.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Akari Asai, Zequi Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.
- [3] Ingeol Baek, Hwan Chang, Byeongjeong Kim, Jimin Lee, and Hwanhee Lee. 2024. Probing-RAG: Self-Probing to Guide Language Models in Selective Document Retrieval. *arXiv preprint arXiv:2410.13339* (2024).
- [4] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B Ashok, and Shashank Shet. 2024. Codeplan: Repository-level coding using llms and planning. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 675–698.
- [5] Avishkar Bhoopchand, Tim Rocktäschel, Earl Barr, and Sebastian Riedel. 2016. Learning python code suggestion with a sparse pointer network. *arXiv preprint arXiv:1611.08307* (2016).
- [6] Zhangqian Bi, Yao Wan, Zheng Wang, Hongyu Zhang, Batu Guan, Fangxin Lu, Zili Zhang, Yulei Sui, Hai Jin, and Xuanhua Shi. 2024. Iterative refinement of project-level code context for precise code generation with compiler feedback. *arXiv preprint arXiv:2403.16792* (2024).
- [7] Tuan-Dung Bui, Duc-Thieu Luu-Van, Thanh-Phat Nguyen, Thu-Trang Nguyen, Son Nguyen, and Hieu Dinh Vo. 2024. Rambo: Enhancing rag-based repository-level method body completion. *arXiv preprint arXiv:2409.15204* (2024).
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [9] Yujia Chen, Cuiyun Gao, Xiaoxue Ren, Yun Peng, Xin Xia, and Michael R Lyu. 2023. API usage recommendation via multi-view heterogeneous graph representation learning. *IEEE Transactions on Software Engineering* 49, 5 (2023), 3289–3304.
- [10] Wei Cheng, Yuhang Wu, and Wei Hu. 2024. Dataflow-guided retrieval augmentation for repository-level code completion. *arXiv preprint arXiv:2405.19782* (2024).
- [11] Ken Deng, Jiaheng Liu, He Zhu, Congnan Liu, Jingxin Li, Jiakai Wang, Peng Zhao, Chenchen Zhang, Yanan Wu, Xueqiao Yin, et al. 2024. R2c2-coder: Enhancing and benchmarking real-world repository-level code completion abilities of code large language models. *arXiv preprint arXiv:2406.01359* (2024).
- [12] Yangruibo Ding, Zijian Wang, Wasi Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. 2023. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *Advances in Neural Information Processing Systems* 36 (2023), 46701–46723.
- [13] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2022. Cocomic: Code completion by jointly modeling in-file and cross-file context. *arXiv preprint arXiv:2212.10007* (2022).
- [14] Yuxin Dong, Shuo Wang, Hongye Zheng, Jiajing Chen, Zhenhong Zhang, and Chihang Wang. 2024. Advanced RAG Models with Graph Structures: Optimizing Complex Knowledge Reasoning and Text Generation. In *2024 5th International Symposium on Computer Engineering and Intelligent Communications (ISCEIC)*. IEEE, 626–630.
- [15] Aryaz Eghbali and Michael Pradel. 2024. De-Hallucinator: Mitigating LLM Hallucinations in Code Generation Tasks via Iterative Grounding. *arXiv preprint arXiv:2401.01701* (2024).
- [16] Robert J Grissom. 1994. Probability of the superior outcome of one treatment over another. *Journal of Applied Psychology* 79, 2 (1994), 314.
- [17] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. Unixcoder: Unified cross-modal pre-training for code representation. *arXiv preprint arXiv:2203.03850* (2022).
- [18] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [19] Lianghong Guo, Yanlin Wang, Ensheng Shi, Wanjuan Zhong, Hongyu Zhang, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. When to stop? towards efficient code generation in llms with excess token prevention. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1073–1085.
- [20] Qi Guo, Xiaohong Li, Xiaofei Xie, Shangqing Liu, Ze Tang, Ruitao Feng, Junjie Wang, Jidong Ge, and Lei Bu. 2024. FT2Ra: A Fine-Tuning-Inspired Approach to Retrieval-Augmented Code Completion. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 313–324.
- [21] Nam Le Hai, Dung Manh Nguyen, and Nghi DQ Bui. 2024. On the Impacts of Contexts on Repository-Level Code Generation. *arXiv preprint arXiv:2406.11927* (2024).
- [22] Shirley Anugrah Hayati, Raphael Olivier, Pravalika Avvaru, Pengcheng Yin, Anthony Tomasic, and Graham Neubig. 2018. Retrieval-based neural code generation. *arXiv preprint arXiv:1808.10025* (2018).
- [23] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131.
- [24] Maliheh Izadi, Roberta Gismondi, and Georgios Gousios. 2022. Codefill: Multi-token code completion by jointly learning from structure and naming sequences. In *Proceedings of the 44th international conference on software engineering*. 401–412.
- [25] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2019. Generalization through memorization: Nearest neighbor language models. *arXiv preprint arXiv:1911.00172* (2019).
- [26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [27] Bolun Li, Zhihong Sun, Tao Huang, Hongyu Zhang, Yao Wan, Ge Li, Zhi Jin, and Chen Lyu. 2024. Ircoco: Immediate rewards-guided deep reinforcement learning for code completion. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 182–203.
- [28] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [29] Yifan Li, Ensheng Shi, Dewu Zheng, Kefeng Duan, Jiachi Chen, and Yanlin Wang. 2024. RepoMinCoder: Improving Repository-Level Code Generation Based on Information Loss Screening. In *Proceedings of the 15th Asia-Pacific Symposium on Internetware*. 229–238.
- [30] Ming Liang, Xiaohong Xie, Gehao Zhang, Xunjin Zheng, Peng Di, Hongwei Chen, Chengpeng Wang, Gang Fan, et al. 2024. Repofuse: Repository-level code completion with fused dual context. *arXiv preprint arXiv:2402.14323* (2024).
- [31] Ming Liang, Xiaohong Xie, Gehao Zhang, Xunjin Zheng, Peng Di, Wei Jiang, Hongwei Chen, Chengpeng Wang, and Gang Fan. 2024. RepoGenix: Dual Context-Aided Repository-Level Code Completion with Language Models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). Association for Computing Machinery, New York, NY, USA, 2466–2467. doi:10.1145/3691620.3695331
- [32] Dianshu Liao, Shidong Pan, Xiaoyu Sun, Xiaoxue Ren, Qing Huang, Zhenchang Xing, Huan Jin, and Qinying Li. 2024. A3-CodGen: A Repository-Level Code Generation Framework for Code Reuse with Local-Aware, Global-Aware, and Third-Party-Library-Aware. *IEEE Transactions on Software Engineering* (2024).
- [33] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [34] Chao Liu, Xin Xia, David Lo, Cuiyun Gao, Xiaohu Yang, and John Grundy. 2021. Opportunities and challenges in code search tools. *ACM Computing Surveys (CSUR)* 54, 9 (2021), 1–40.
- [35] Tianyang Liu, Canwen Xu, and Julian McAuley. 2023. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091* (2023).
- [36] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. 2024. Graphcoder: Enhancing repository-level code completion via code context graph-based retrieval and language model. *arXiv preprint arXiv:2406.07003* (2024).
- [37] Shuai Lu, Nan Duan, Hojae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. 2022. Reacc: A retrieval-augmented code completion framework. *arXiv preprint arXiv:2203.07722* (2022).

- [38] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60.
- [39] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2024. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. *arXiv preprint arXiv:2410.14684* (2024).
- [40] Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. *arXiv preprint arXiv:2108.11601* (2021).
- [41] Huy N Phan, Hoang N Phan, Tien N Nguyen, and Nghi DQ Bui. 2024. Repohyper: Better context retrieval is all you need for repository-level code completion. *arXiv e-prints* (2024), arXiv–2403.
- [42] S Jansi Rani, SG Deepika, D Devdharshini, and Harini Ravindran. 2024. Augmenting Code Sequencing with Retrieval-Augmented Generation (RAG) for Context-Aware Code Synthesis. In *2024 First International Conference on Software, Systems and Information Technology (SSITCON)*. IEEE, 1–7.
- [43] Veselin Raychev, Martin Vechev, and Eran Yahav. 2014. Code completion with statistical language models. In *Proceedings of the 35th ACM SIGPLAN conference on programming language design and implementation*. 419–428.
- [44] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [45] Jiho Shin and Jaechang Nam. 2021. A survey of automatic code generation from natural language. *Journal of Information Processing Systems* 17, 3 (2021), 537–555.
- [46] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. Intellicode compose: Code generation using transformer. In *Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 1433–1443.
- [47] Ze Tang, Jidong Ge, Shangqing Liu, Tingwei Zhu, Tongtong Xu, Liguang Huang, and Bin Luo. 2023. Domain adaptive code completion via language models and decoupled domain databases. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 421–433.
- [48] Jicheng Wang, Yifeng He, and Hao Chen. 2024. RepoGenReflex: Enhancing Repository-Level Code Completion with Verbal Reinforcement and Retrieval-Augmented Generation. *arXiv preprint arXiv:2409.13122* (2024).
- [49] Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. 2024. Toolgen: Unified tool retrieval and calling via generation. *arXiv preprint arXiv:2410.03439* (2024).
- [50] Yanlin Wang and Hui Li. 2021. Code completion by modeling flattened abstract syntax trees as graphs. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 14015–14023.
- [51] Yanlin Wang, Yanli Wang, Daya Guo, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. RlCoder: Reinforcement learning for repository-level code completion. *arXiv preprint arXiv:2407.19487* (2024).
- [52] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. Repoformer: Selective retrieval for repository-level code completion. *arXiv preprint arXiv:2403.10059* (2024).
- [53] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
- [54] Jian Yang, Jiajun Zhang, Jiayi Yang, Ke Jin, Lei Zhang, Qiyao Peng, Ken Deng, Yibo Miao, Tianyu Liu, Zeyu Cui, Binyuan Hui, and Junyang Lin. 2024. ExecRepoBench: Multi-level Executable Code Completion Evaluation. *CoRR abs/2412.11990* (2024). doi:10.48550/ARXIV.2412.11990 arXiv:2412.11990
- [55] Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2023. Making retrieval-augmented language models robust to irrelevant context. *arXiv preprint arXiv:2310.01558* (2023).
- [56] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.
- [57] Tian Yu, Shaolei Zhang, and Yang Feng. 2024. Auto-rag: Autonomous retrieval-augmented generation for large language models. *arXiv preprint arXiv:2411.19443* (2024).
- [58] Daoguang Zan, Bei Chen, Yongshun Gong, Junzhi Cao, Fengji Zhang, Bingchao Wu, Bei Guan, Yilong Yin, and Yongji Wang. 2023. Private-library-oriented code generation with large language models. *arXiv preprint arXiv:2307.15370* (2023).
- [59] Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. CERT: continual pre-training on sketches for library-oriented code generation. *arXiv preprint arXiv:2206.06888* (2022).
- [60] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570* (2023).
- [61] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. *arXiv preprint arXiv:2401.07339* (2024).
- [62] Wenrui Zhang, Tiehang Fu, Ting Yuan, Ge Zhang, Dong Chen, and Jie Wang. 2024. A Lightweight Framework for Adaptive Retrieval In Code Completion With Critique Model. *arXiv preprint arXiv:2406.10263* (2024).
- [63] Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. Codebertscore: Evaluating code generation with pretrained models of code. *arXiv preprint arXiv:2302.05527* (2023).
- [64] Shuyan Zhou, Uri Alon, Frank F Xu, Zhiruo Wang, Zhengbao Jiang, and Graham Neubig. 2022. Docprompting: Generating code by retrieving the docs. *arXiv preprint arXiv:2207.05987* (2022).
- [65] Yuqi Zhu, Jia Li, Ge Li, Yunfei Zhao, Zhi Jin, and Hong Mei. 2024. Hot or cold? adaptive temperature sampling for code generation with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 437–445.